VULNERABILITY TESTING / LAB REPORT

FILE INCLUSION

Web Application Penetration Testing – Internship Report

Organization: IqraSec Academy
Platform: TryHackMe

COVER PAGE

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT)
Topic	File Inclusion
Platform	TryHackMe
Number of Labs	1
Completion Date	25-10-2026

This report documents the objective, vulnerability concept, testing methodology, evidence, impact, remediation, and learning outcomes of the File Inclusion lab.

Lab 1: File Inclusion

1. Lab Name

File Inclusion

Platform: TryHackMe
Room URL: https://tryhackme.com/room/fileinc

2. Objective

The objective of this lab is to understand file inclusion vulnerabilities in web applications and learn how insecure file-handling logic can expose sensitive information or execute unintended code.

The main objectives are:

- Understand Local File Inclusion (LFI).
- Understand Remote File Inclusion (RFI).

- Learn how user-controlled input may affect file paths.
- Identify potentially vulnerable file parameters.
- Understand directory traversal and its relationship to file inclusion.
- Learn the impact of insecure file inclusion.
- Study secure file-handling and remediation techniques.

3. Vulnerability / Concept

What is File Inclusion?

File Inclusion is a vulnerability that occurs when an application uses user-controlled input to determine which file to load or include without sufficient validation.

Depending on the application and its configuration, an attacker may be able to access unintended files or cause an application to process a file that should not be permitted.

Types of File Inclusion

1. Local File Inclusion (LFI)

LFI occurs when an application improperly includes or reads a file from the local server's filesystem.

Potential consequences include exposure of configuration files, application source code, or other sensitive information, depending on the application's permissions and implementation.

2. Remote File Inclusion (RFI)

RFI occurs when an application is configured to include a file from a remote location and an attacker can influence the remote resource being included.

Successful exploitation depends on the application's implementation and server configuration. RFI can potentially lead to code execution in vulnerable environments.

3. Directory Traversal

Directory traversal occurs when an attacker manipulates a file path to access locations outside the intended directory. It can overlap with LFI, but the two terms describe different aspects of a vulnerability.

Common Indicators

- File paths or filenames are accepted from user-controlled parameters.
- Application pages load different files based on request parameters.
- Unexpected filesystem paths can be accessed.
- File access controls are missing or incorrectly implemented.
- The application relies on string filtering instead of secure path validation.

4. Testing Approach

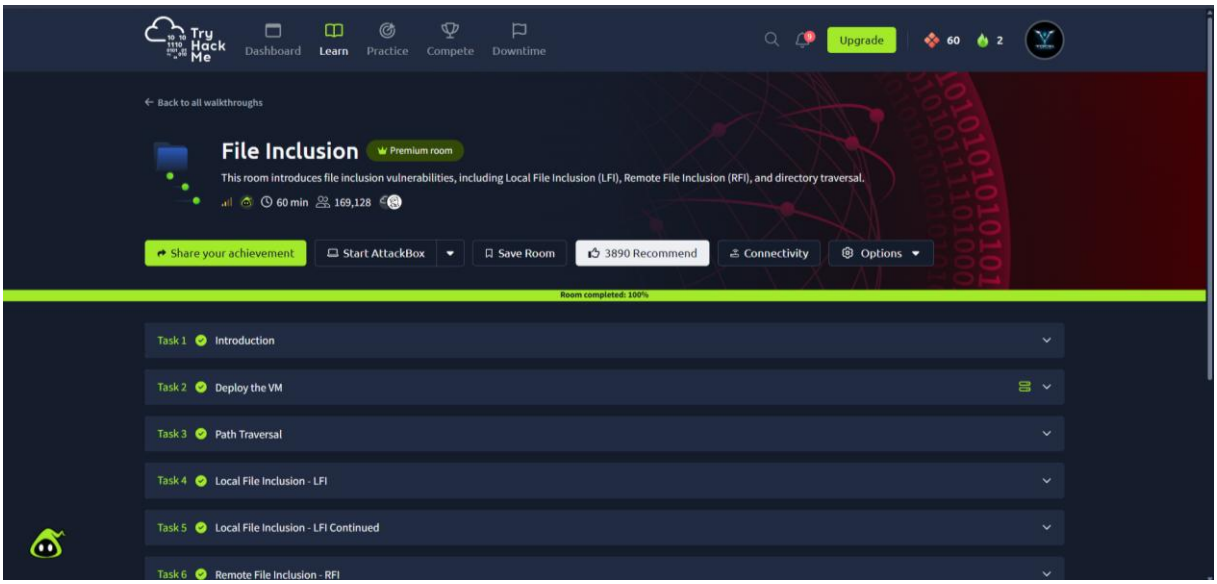
The following workflow can be used in the authorized TryHackMe lab environment.

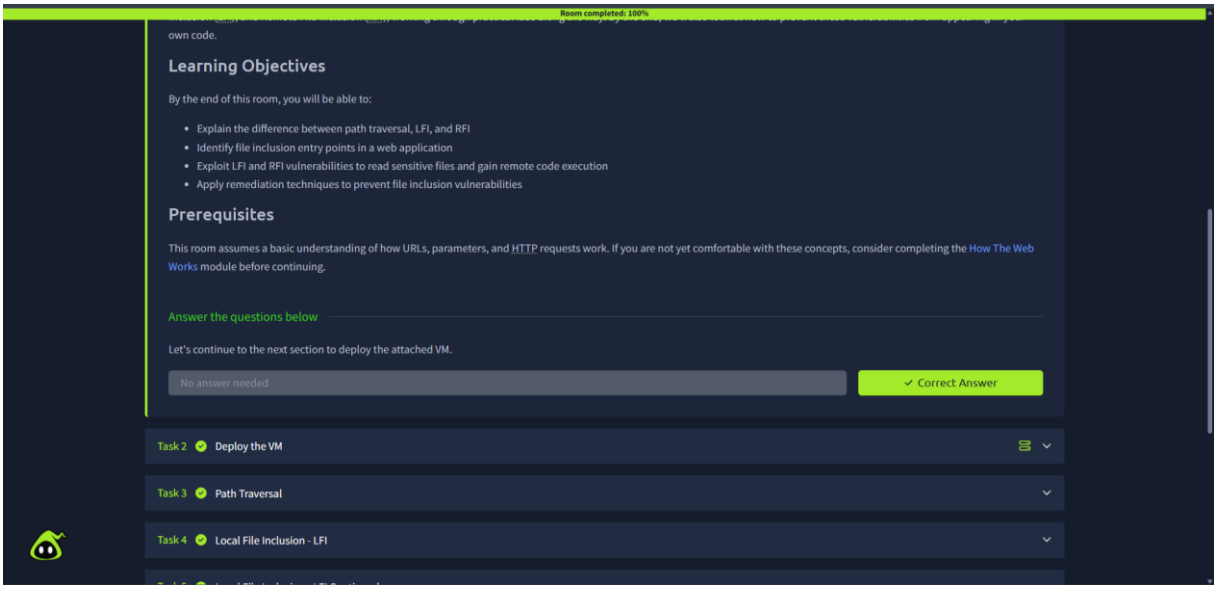
1. Open the File Inclusion room on TryHackMe.

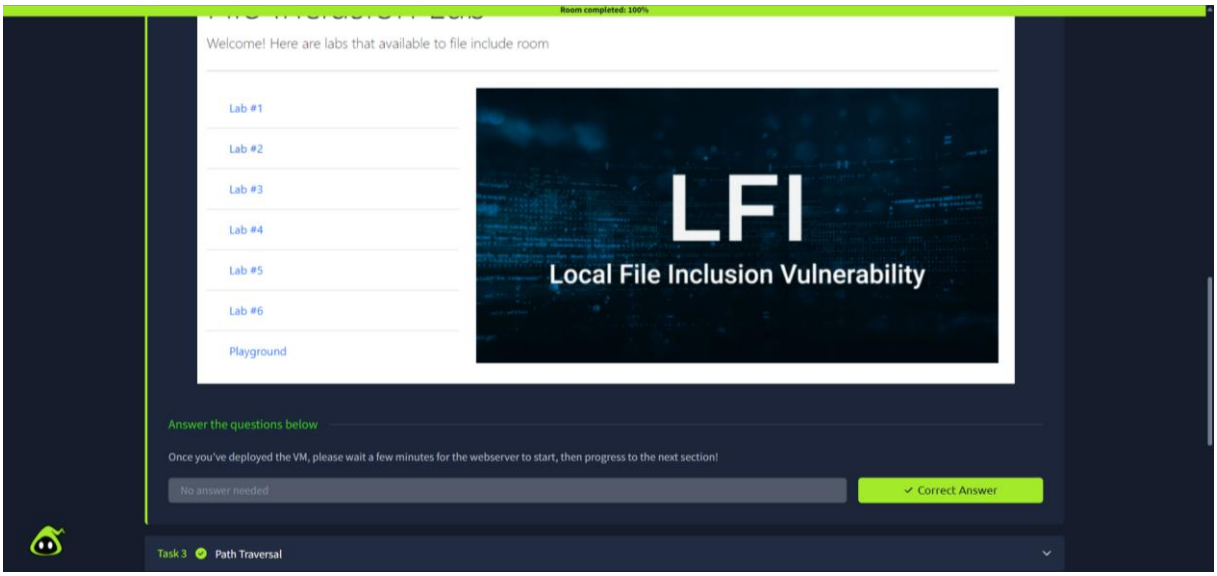
2. Read the room instructions and learning objectives.
3. Start the provided lab environment if required.
4. Open the target web application.
5. Identify pages or features that load files dynamically.
6. Observe URL parameters, form fields, and requests that may specify a filename or path.
7. Establish the application's normal behavior using valid input supplied by the lab.
8. Identify how the application handles file paths and filenames.
9. Review the lab material on Local File Inclusion.
10. Follow the lab's instructions to test whether a local file can be accessed through the relevant functionality.
11. Observe the application's response and record any errors or returned content.
12. Review the lab material on directory traversal.
13. Test the traversal technique specified by the lab against the authorized target.
14. Determine whether the application restricts access to the intended directory.
15. Review the lab material on Remote File Inclusion, if covered.
16. Determine whether the application accepts remote file references and whether the relevant server configuration permits inclusion.
17. Record the exact requests and outcomes observed during the exercise.
18. Capture screenshots showing the relevant test and its result.
19. Compare the observed behavior with secure file-handling practices.
20. Document confirmed findings, potential impact, and recommended remediation.

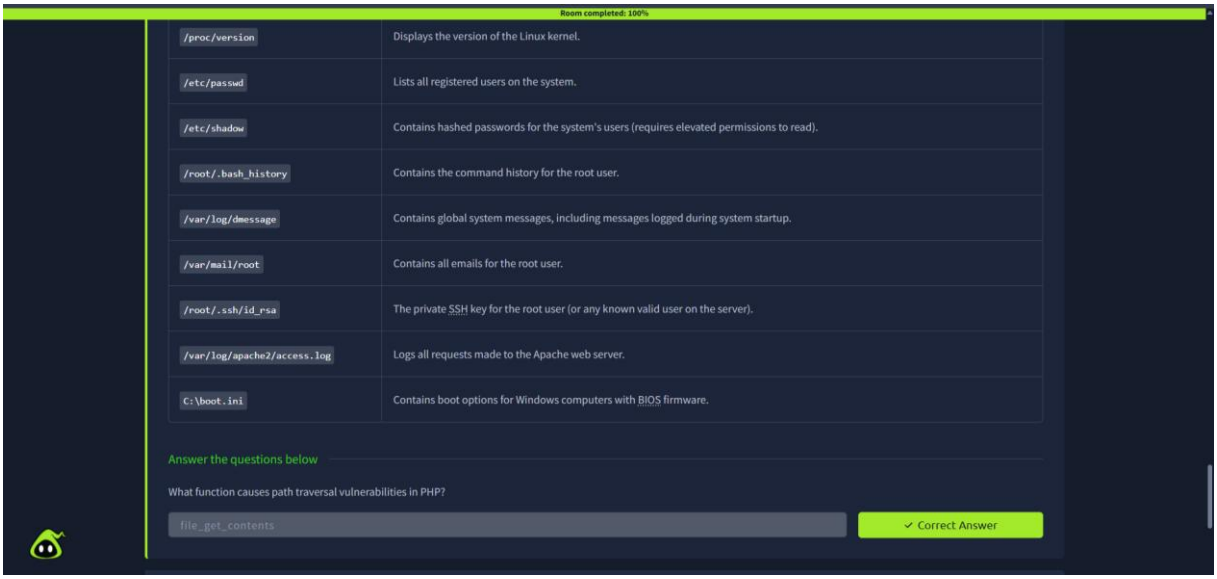
Testing note: Perform tests only against the assigned lab environment. Do not claim successful exploitation unless the observed result supports that conclusion.

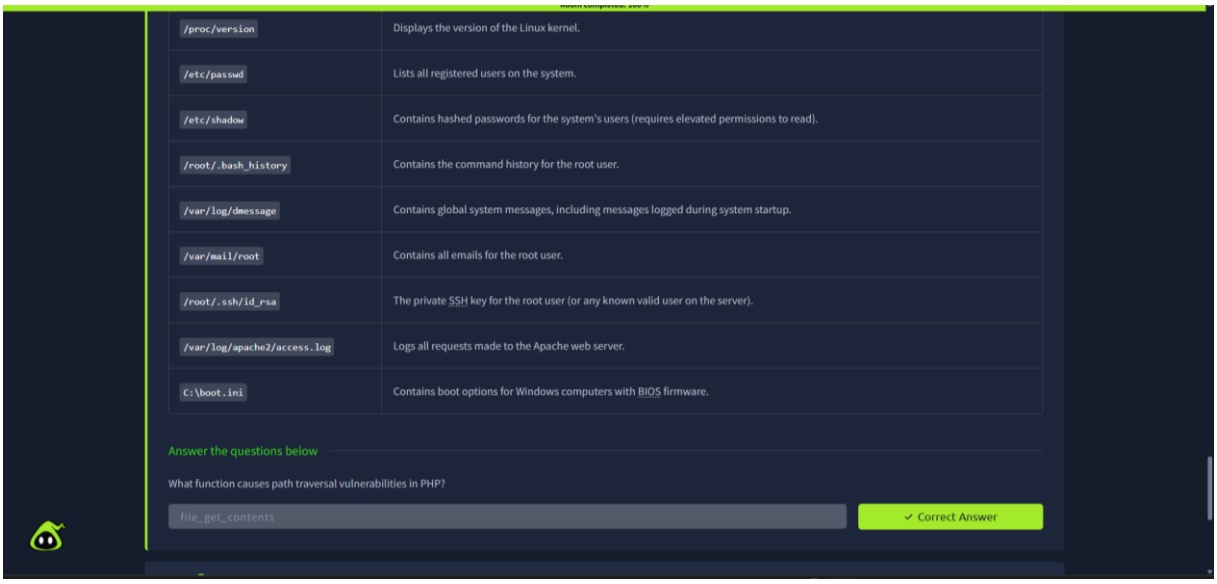
5. Evidence / Screenshots

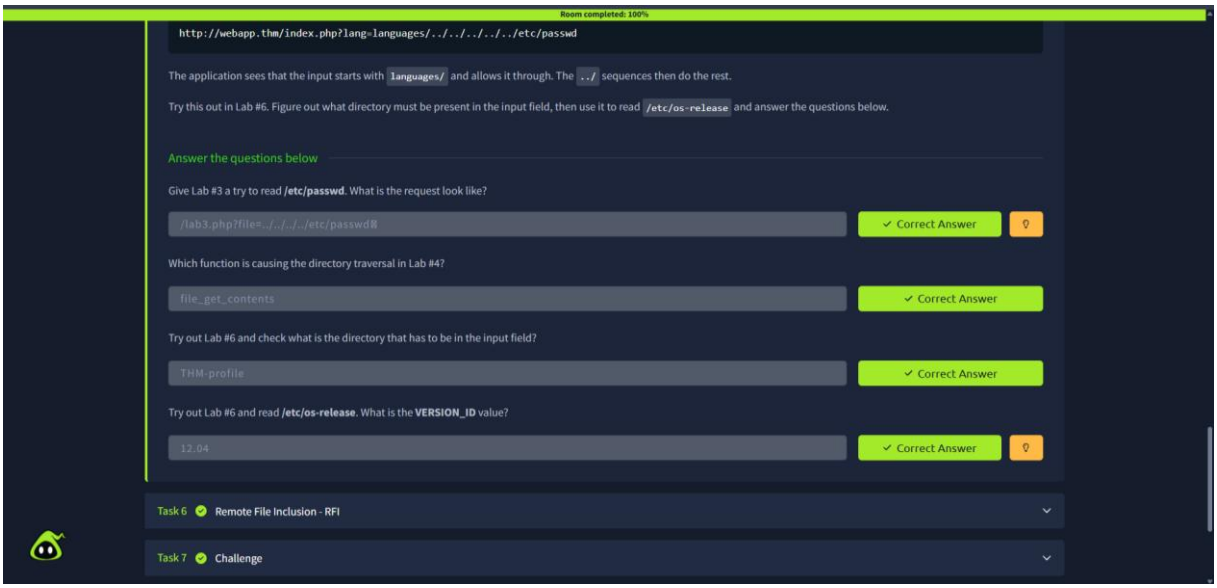

Try Hack Me
Dashboard
Learn
Practice
Compete
Downtime
Upgrade
60
2
Back to all walkthroughs
File Inclusion
Premium room
This room introduces file inclusion vulnerabilities, including Local File Inclusion (LFI), Remote File Inclusion (RFI), and directory traversal.
60 min
169,128
Share your achievement
Start AttackBox
Save Room
3890 Recommend
Connectivity
Options
Room completed: 100%
Task 1 Introduction
Task 2 Deploy the VM
Task 3 Path Traversal
Task 4 Local File Inclusion - LFI
Task 5 Local File Inclusion - LFI Continued
Task 6 Remote File Inclusion - RFI

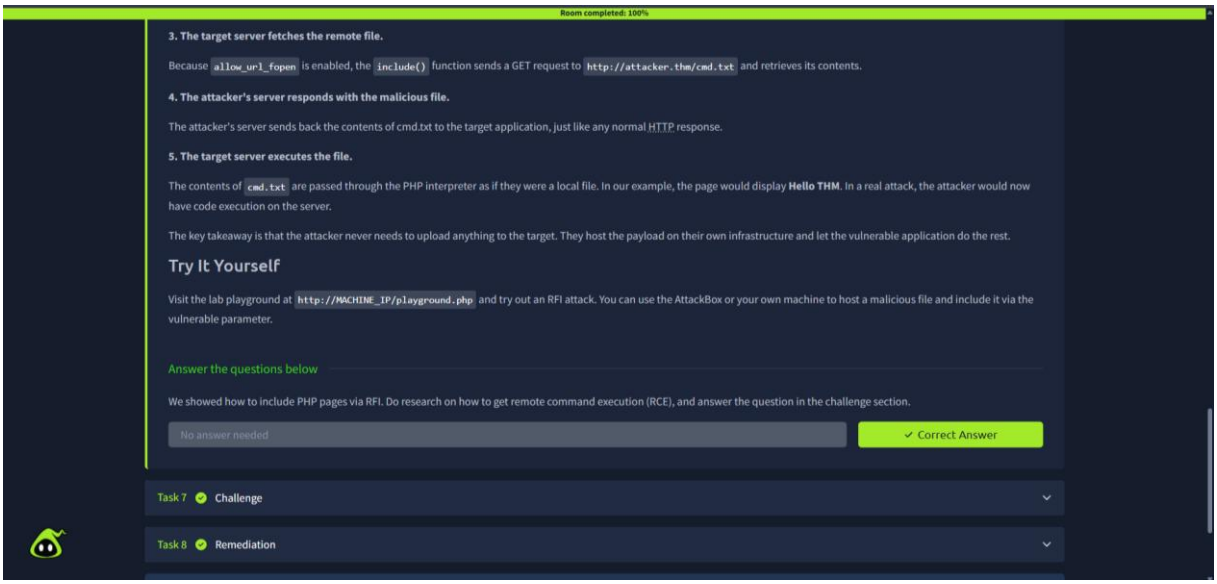

Room completed: 100%
own code.
Learning Objectives
By the end of this room, you will be able to:
Explain the difference between path traversal, LFI, and RFI
Identify file inclusion entry points in a web application
Exploit LFI and RFI vulnerabilities to read sensitive files and gain remote code execution
Apply remediation techniques to prevent file inclusion vulnerabilities
Prerequisites
This room assumes a basic understanding of how URLs, parameters, and HTTP requests work. If you are not yet comfortable with these concepts, consider completing the How The Web Works module before continuing.
Answer the questions below
Let's continue to the next section to deploy the attached VM.
No answer needed
✓ Correct Answer
Task 2 Deploy the VM
Task 3 Path Traversal
Task 4 Local File Inclusion - LFI

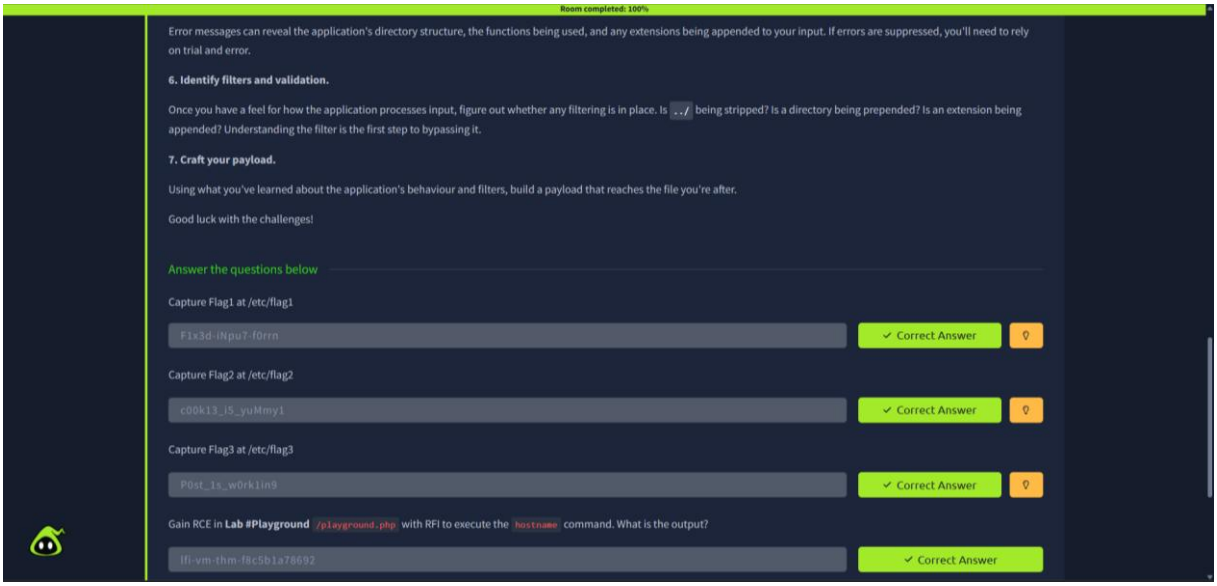

Room completed: 100%
Welcome! Here are labs that available to file include room
Lab #1
Lab #2
Lab #3
Lab #4
Lab #5
Lab #6
Playground
LFI
Local File Inclusion Vulnerability
Answer the questions below
Once you've deployed the VM, please wait a few minutes for the webserver to start, then progress to the next section!
No answer needed
✓ Correct Answer
Task 3 Path Traversal

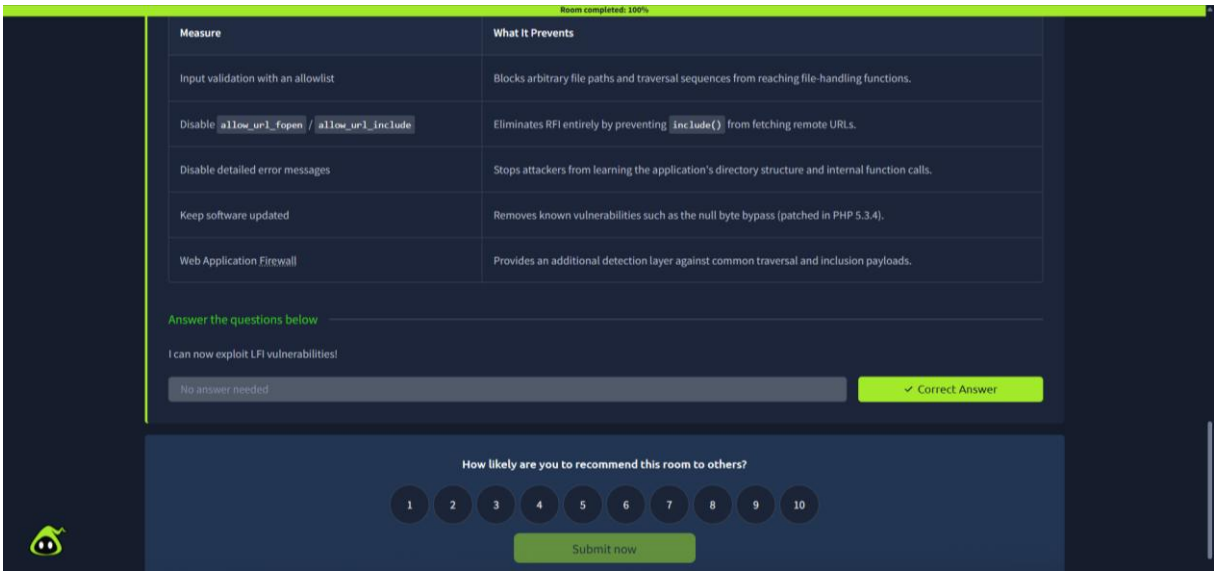

Room completed: 100%
/proc/version
Displays the version of the Linux kernel.
/etc/passwd
Lists all registered users on the system.
/etc/shadow
Contains hashed passwords for the system's users (requires elevated permissions to read).
/root/.bash_history
Contains the command history for the root user.
/var/log/dmessage
Contains global system messages, including messages logged during system startup.
/var/mail/root
Contains all emails for the root user.
/root/.ssh/id_rsa
The private SSH key for the root user (or any known valid user on the server).
/var/log/apache2/access.log
Logs all requests made to the Apache web server.
C:\boot.ini
Contains boot options for Windows computers with BIOS firmware.
Answer the questions below
What function causes path traversal vulnerabilities in PHP?
file_get_contents
✓ Correct Answer


/proc/version
Displays the version of the Linux kernel.
/etc/passwd
Lists all registered users on the system.
/etc/shadow
Contains hashed passwords for the system's users (requires elevated permissions to read).
/root/.bash_history
Contains the command history for the root user.
/var/log/dmessage
Contains global system messages, including messages logged during system startup.
/var/mail/root
Contains all emails for the root user.
/root/.ssh/id_rsa
The private SSH key for the root user (or any known valid user on the server).
/var/log/apache2/access.log
Logs all requests made to the Apache web server.
C:\boot.ini
Contains boot options for Windows computers with BIOS firmware.
Answer the questions below
What function causes path traversal vulnerabilities in PHP?
file_get_contents
✓ Correct Answer


Room completed: 100%
http://webapp.thm/index.php?lang=languages/../../../../etc/passwd
The application sees that the input starts with languages/ and allows it through. The ../ sequences then do the rest.
Try this out in Lab #6. Figure out what directory must be present in the input field, then use it to read /etc/os-release and answer the questions below.
Answer the questions below
Give Lab #3 a try to read /etc/passwd. What is the request look like?
/lab3.php?file=../../../../etc/passwd
✓ Correct Answer
Which function is causing the directory traversal in Lab #4?
file_get_contents
✓ Correct Answer
Try out Lab #6 and check what is the directory that has to be in the input field?
THM-profile
✓ Correct Answer
Try out Lab #6 and read /etc/os-release. What is the VERSION_ID value?
12.04
✓ Correct Answer
Task 6 Remote File Inclusion - RFI
Task 7 Challenge


Room completed: 100%
3. The target server fetches the remote file.
Because allow_url_fopen is enabled, the include() function sends a GET request to http://attacker.thm/cmd.txt and retrieves its contents.
4. The attacker's server responds with the malicious file.
The attacker's server sends back the contents of cmd.txt to the target application, just like any normal HTTP response.
5. The target server executes the file.
The contents of cmd.txt are passed through the PHP interpreter as if they were a local file. In our example, the page would display Hello THM. In a real attack, the attacker would now have code execution on the server.
The key takeaway is that the attacker never needs to upload anything to the target. They host the payload on their own infrastructure and let the vulnerable application do the rest.
Try It Yourself
Visit the lab playground at http://MACHINE_IP/playground.php and try out an RFI attack. You can use the AttackBox or your own machine to host a malicious file and include it via the vulnerable parameter.
Answer the questions below
We showed how to include PHP pages via RFI. Do research on how to get remote command execution (RCE), and answer the question in the challenge section.
No answer needed
✓ Correct Answer
Task 7 Challenge
Task 8 Remediation


Room completed: 100%
Error messages can reveal the application's directory structure, the functions being used, and any extensions being appended to your input. If errors are suppressed, you'll need to rely on trial and error.
6. Identify filters and validation.
Once you have a feel for how the application processes input, figure out whether any filtering is in place. Is ../ being stripped? Is a directory being prepended? Is an extension being appended? Understanding the filter is the first step to bypassing it.
7. Craft your payload.
Using what you've learned about the application's behaviour and filters, build a payload that reaches the file you're after.
Good luck with the challenges!
Answer the questions below
Capture Flag1 at /etc/flag1
F1x3d-iNpu7-f0rrn
✓ Correct Answer
Capture Flag2 at /etc/flag2
c00k13_iS_yuMmy1
✓ Correct Answer
Capture Flag3 at /etc/flag3
P0st_1s_w0rk1in9
✓ Correct Answer
Gain RCE in Lab #Playground /playground.php with RFI to execute the hostname command. What is the output?
lfi-vm-thm-f8c5b1a78692
✓ Correct Answer



6. Impact

If exploitable, file inclusion vulnerabilities may have the following consequences:

- **Sensitive information disclosure:** Application configuration, source code, or other files may be exposed.
- **Credential exposure:** Files containing credentials or secrets may be accessible, depending on server permissions.
- **Application compromise:** In some environments, file inclusion may contribute to further exploitation.
- **Remote code execution:** Certain vulnerable configurations may allow remote code to be included and executed.
- **Confidentiality and integrity risks:** Sensitive information may be disclosed or application behavior may be affected.
- **Business impact:** Successful exploitation may result in data exposure, service disruption, or reputational damage.

The actual severity depends on the vulnerability type, the server's configuration, file permissions, and the impact demonstrated during testing.

7. Remediation

The following measures help prevent file inclusion vulnerabilities.

1. Avoid user-controlled file paths

Where possible, use predefined identifiers mapped to approved files instead of accepting arbitrary paths from users.

2. Use allowlists

Restrict file selection to explicitly permitted files or directories.

3. Canonicalize and validate paths

Resolve paths to their canonical form and verify that the final path remains within the intended directory.

4. Disable unnecessary remote inclusion features

Disable remote file inclusion capabilities when they are not required by the application.

5. Use least-privilege permissions

Ensure that the web server account cannot access files or directories beyond its operational requirements.

6. Avoid relying on blacklist filtering

Simple string filters can often be bypassed. Use secure file-handling APIs and robust validation instead.

7. Protect sensitive files

Store configuration files, secrets, and credentials outside publicly accessible directories where practical, and restrict filesystem permissions.

8. Handle errors securely

Avoid exposing filesystem paths, source code, or sensitive internal details in error messages returned to users.

9. Keep application components updated

Apply security updates to the web framework, runtime, server, and relevant dependencies.

10. Perform regular security testing

Test file-handling features for path traversal, local file inclusion, and other relevant file access weaknesses.

8. What You Learned

The main learning outcomes of this lab are:

- Understanding the concept of file inclusion vulnerabilities.
- Distinguishing Local File Inclusion from Remote File Inclusion.
- Understanding the relationship between directory traversal and LFI.
- Identifying user-controlled file paths and filename parameters.
- Observing how an application handles unexpected file input.
- Understanding the security implications of exposed files and insecure file-handling logic.
- Learning how allowlists, path validation, and least privilege reduce risk.
- Documenting test methodology, evidence, findings, and remediation recommendations.

Update these learning outcomes according to the topics and tasks you actually completed in the room.

Conclusion

File inclusion vulnerabilities can expose sensitive files or, in certain configurations, contribute to more serious compromise. Secure file-handling requires strict control over which files an application can access, careful validation of file paths, appropriate filesystem permissions, and disabling unnecessary remote inclusion functionality.

This report provides a structured approach to examining file inclusion behavior in an authorized lab environment. The final assessment should be based on actual observations, supporting screenshots, and the recorded completion status of the TryHackMe room.

Report Prepared By: Rohit Kushwah
Organization: IqraSec Academy
Internship: Web Application Penetration Testing (WAPT)